
PQ-SIFT: Projected-Query Sign-based Filtering of Tokens for Efficient Long-Context Attention

Korea University COSE461 Final Project

Minseok Choi

Department of Computer Science
Korea University
Team 31
kw4365@gmail.com

Geonhee Jang

Department of Data Science
Korea University
Team 31
singlegun2@korea.ac.kr

Abstract

Autoregressive long-context inference is bottlenecked by more than the final QK-softmax-V arithmetic. At every decode step, the model must repeatedly decide *which* past tokens to revisit and then read the corresponding KV cache entries. Full attention is accurate but reads the entire cache; low-rank sparse methods such as LOKI reduce final attention work but still pay dense projected scoring, sorting, and scattered gathering overhead. We propose **PQ-SIFT**, a query-aware sparse retrieval method that replaces dense low-dimensional ranking with **projected-query sign/range filtering**. Offline, keys are projected into a PCA subspace, summarized by compact sign/range metadata, and grouped into 64-token blocks. Online, the query is projected once, unlikely blocks are rejected from metadata, and exact attention is computed only on fetched candidates. The bucketed version aligns sparse retrieval with contiguous block/page reads. On LLaMA-3.1-8B decode-style experiments, PQ-SIFT-BUCKET gives a clearer quality-latency win at long context: at 96K tokens, it raises mass recall from 0.561 to 0.865 and output cosine from 0.678 to 0.936 compared with a LOKI $A = 64$, $K = 1024$ baseline, while reducing average token latency from 0.368 ms to 0.180 ms. In other words, it keeps substantially more of the useful attention signal while roughly halving measured decode latency in this setting. These results suggest that practical long-context sparse attention should optimize retained attention mass, candidate generation, and page-friendly KV access together rather than treating them as separate problems.

1 Introduction

Transformer language models compare each generated query with all previous keys [1]. During decoding, the context grows token by token, so every step repeatedly reads from the KV cache. This creates a systems bottleneck: even if the mathematical attention operation is simple, long-context inference becomes dominated by memory traffic, cache layout, and the overhead required to decide which past tokens should be attended to.

A common response is sparse attention. However, not every sparse method improves end-to-end latency. Fixed recent-window methods are cheap but lose distant information. Low-rank Top- K retrieval methods, including Loki-style baselines, can keep important tokens but still perform dense projected scans and sorting over the entire context. In other words, they reduce the final QK-softmax-V computation but leave a candidate-generation path that is itself expensive.

This report studies a different design point: *coarse, block-friendly candidate admission*. PQ-SIFT projects keys and queries into a small PCA space but avoids dense continuous ranking at decode time.

Instead, it uses sign/range metadata to reject unlikely KV blocks before fetching full keys and values. The optimized PQ-SIFT-Bucket variant stores candidates in 64-token blocks so that the selected context can be fetched as contiguous chunks rather than scattered token indices.

Our contributions are threefold. First, we formulate decode-time sparse attention as a joint quality-and-systems problem: a method is not useful if it reduces final arithmetic but loses most attention mass or spends the saved time on candidate generation. Second, we propose a projected-query sign/range filtering pipeline that changes candidate generation from dense Top- K ranking to metadata-based block filtering. This design intentionally favors block admission and coalesced reads over token-by-token scattered selection. Third, we analyze the trade-off across context length, PCA axes, inclusion ratio r , layer sensitivity, perplexity sanity checks, and latency components using the final twelve visual panels. The goal is not only to report that PQ-SIFT is faster, but also to explain *why* the speedup appears and under which operating points the quality remains acceptable.

2 Related Work

Efficient attention approximations. Reformer uses locality-sensitive hashing to reduce attention complexity [2]; Longformer and BigBird introduce structured sparse patterns for long documents [3, 4]; Performer approximates softmax attention with random features [5]. These methods motivate the broad goal of reducing attention cost, but they primarily address architectural or training-time attention. Our work focuses on decode-time KV retrieval and memory movement.

IO-aware exact attention and KV memory systems. FlashAttention shows that attention speed depends strongly on HBM/SRAM IO, not only on FLOPs [6]. PagedAttention/vLLM stores KV cache in fixed-size blocks to improve serving-time memory management [7]. PQ-SIFT-Bucket follows this systems perspective: sparse attention should be designed so that the candidate set maps naturally onto memory pages and block fetches.

KV-cache compression and retention policies. H₂O, StreamingLLM, SnapKV, and DuoAttention reduce the amount of KV state that must remain active by retaining heavy hitters, attention sinks, observation-window summaries, or retrieval heads [8–11]. These methods decide what to keep in cache. In contrast, PQ-SIFT assumes the full cache may remain available and instead decides which blocks to read for the current query.

Query-aware sparse retrieval. Quest selects pages by estimating page importance from key bounds [12]; SparQ reduces bandwidth by using partial query/key dimensions before fetching values [13]; Loki ranks candidates in a low-rank key space and then runs exact attention on selected tokens [14]. PQ-SIFT is closest to this query-aware family because the active candidate set depends on the current query. The main difference is the candidate-generation mechanism. Rather than computing a score for every token in a low-dimensional space and sorting the results, PQ-SIFT asks a coarser question: which blocks are geometrically plausible under the query’s projected sign/range pattern? This change is deliberately system-oriented. It may admit a few extra tokens, but it avoids a dense scan-and-sort path and makes the remaining memory access more page friendly.

3 Approach

Figure 1 gives the main system view, so this section is intentionally compact. For a decode query q , exact attention over keys $K = \{k_i\}_{i=1}^n$ and values $V = \{v_i\}_{i=1}^n$ computes

$$\alpha_i = \frac{\exp(q^\top k_i / \sqrt{d})}{\sum_{j=1}^n \exp(q^\top k_j / \sqrt{d})}, \quad o = \sum_{i=1}^n \alpha_i v_i. \quad (1)$$

A sparse retrieval method selects a candidate set $\mathcal{C}(q)$ and computes exact attention only within that set. We measure fidelity using attention mass recall

$$\text{MR}(q) = \sum_{i \in \mathcal{C}(q)} \alpha_i, \quad (2)$$

with output cosine similarity as an additional check against exact attention.

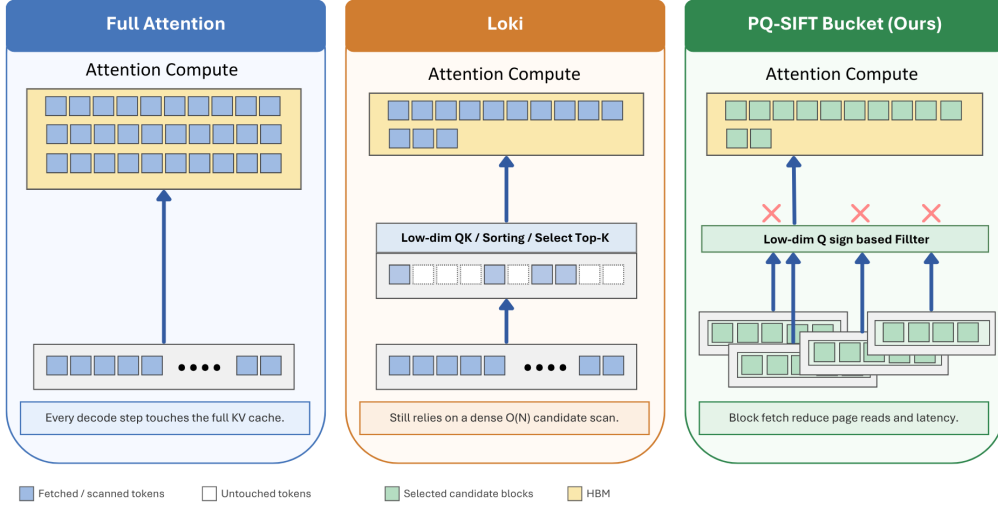


Figure 1: System architecture. Full attention reads the entire KV cache at every step. Loki reduces the final attention set but still performs dense low-dimensional QK scoring, sorting, and Top- K selection. PQ-SIFT-Bucket uses low-dimensional query-sign filtering over block metadata, fetches selected candidate blocks, and computes exact attention only on those candidates.

Offline metadata construction. We fit a PCA basis $W_{\text{PCA}} \in \mathbb{R}^{d \times A}$ from cached key vectors and compute projected keys

$$z_i = k_i W_{\text{PCA}} \in \mathbb{R}^A. \quad (3)$$

Instead of using these projected coordinates only for dense online ranking, PQ-SIFT stores compact sign/range summaries. PQ-SIFT-Bucket groups tokens into fixed 64-token blocks and stores per-block sign and range metadata. This grouping is important because it converts sparse retrieval from scattered token gather into coalesced block fetch.

The reason sign-only metadata can still be effective is that PCA projection preserves the dominant variance directions of the key space. Although PQ-SIFT discards exact projected magnitudes and retains only coarse sign/range information, the sign pattern identifies the orthant in which a token lies in the projected space. Tokens in the same or nearby orthants often share a coarse directional relationship to the query, so sign agreement can act as a cheap proxy for whether a block is geometrically plausible before loading full keys. In this sense, PQ-SIFT does not try to approximate exact attention scores directly; it uses PCA signs as a lightweight admission test and leaves the final QK-softmax-V computation exact on the surviving candidates.

Online projected-query filtering. At decode time, the query is projected once, $z_q = q^\top W_{\text{PCA}}$. Its signs and ranges are compared with block metadata. Blocks that fail the test are skipped without loading full keys and values. A simplified candidate rule is

$$\mathcal{C}(q) = \{i : \text{block}(i) \in \mathcal{B}_{\text{pass}}(z_q, A, r)\} \cup \mathcal{R}, \quad (4)$$

where A is the number of PCA axes, r controls how much opposite-sign/range-expanded mass is admitted, and $\mathcal{R}$ denotes optional recent-token or sensitive-layer fallback. Larger r increases quality but also increases candidate count.

4 Experiments

4.1 Data

We use LLaMA-3.1-8B as the backbone model [15]. The PCA basis is trained offline on approximately **20 million tokens from WikiText-103** [16]. This large corpus is used only to learn a stable projection

Algorithm 1 PQ-SIFT-Bucket sparse retrieval at one decode step

- 1: **Input:** query q , KV cache K, V , PCA basis W_{PCA} , block size $B = 64$, axes A , ratio r
 - 2: Project query: $z_q \leftarrow q^\top W_{\text{PCA}}[:, 1 : A]$
 - 3: Read sign/range metadata for each 64-token block
 - 4: Select blocks whose metadata passes the query-side sign/range test
 - 5: Fetch K, V only for selected blocks using contiguous chunks
 - 6: Return exact attention $\text{softmax}(qK_c^\top / \sqrt{d})V_c$
-

basis for the key space; it is not the text on which the final reported decode metrics are evaluated. The actual decode-style evaluation uses the **WikiText-2 test set**, following the experimental setup of the Loki paper. Separating basis learning from evaluation is important because it prevents the reported attention fidelity from being a direct artifact of the same token stream used to fit PCA. WikiText-2 test is therefore used to measure attention mass recall, output cosine, candidate selection, page-read behavior, latency, and the perplexity sanity check described below. The scaling plots cover context lengths from 2K to 96K.

4.2 Evaluation method

We evaluate both model-side fidelity and system-side cost. Fidelity is measured by attention mass recall MR and output cosine similarity relative to full attention. Mass recall answers whether the selected candidates contain the tokens that the exact distribution would have emphasized, while cosine similarity checks whether the final value aggregation remains close to the full-attention output. Efficiency is measured by MAC ratio, page-read ratio, average token latency, and component-level latency. This combination is necessary because a method can look efficient under final-attention FLOPs but still pay a large online retrieval, sorting, or gather overhead. We therefore interpret each result as a trade-off among three quantities: retained attention signal, number of candidates admitted, and the cost of moving KV blocks through memory.

4.3 Experimental details

Baselines include full attention, recent-window sparse attention, and Loki-style low-rank sparse attention with $A = 32$ or $A = 64$ and $K = 1024$ candidates. Our variants are PQ-SIFT Range and PQ-SIFT-Bucket. For hardware, experiments up to 8K context were run on an **A100 GPU**, while the 96K long-context run was run on a **B200 GPU**. Therefore, absolute latency at very long context should be interpreted with the hardware setting in mind; the most reliable conclusions are relative trends within each measurement setting and the breakdown of where time is spent.

We also ran a decode-style perplexity sanity check on the **WikiText-2 test set**. In this experiment, each evaluation uses a 4K-token context; query positions are selected every 128 tokens, and the procedure is repeated for 2,000 query evaluations. This setting follows the spirit of Loki-style sparse-attention evaluation: instead of evaluating only a single hand-picked query, it repeatedly probes many decode positions so that the reported perplexity reflects the stability of the candidate-selection rule across the test stream.

4.4 Results

Figure 2 summarizes the main scaling result. Full attention is the quality reference, but it keeps MAC and page-read ratios near 1.0 because every token remains active. Recent-window attention has the opposite behavior: it is cheap, but its quality quickly degrades when important information falls outside the window. Loki occupies the middle ground by selecting Top- K tokens in a low-rank space. However, as context grows, a fixed K covers a smaller fraction of the full attention distribution, so mass recall and output quality become increasingly sensitive to the ranking step. PQ-SIFT and especially PQ-SIFT-Bucket provide a better long-context operating point because they keep the retrieval rule query-aware while making memory access block-friendly. At 96K, the most important observation is not just that latency is lower, but that the latency reduction comes together with *higher* retained attention mass than the comparable Loki baseline.

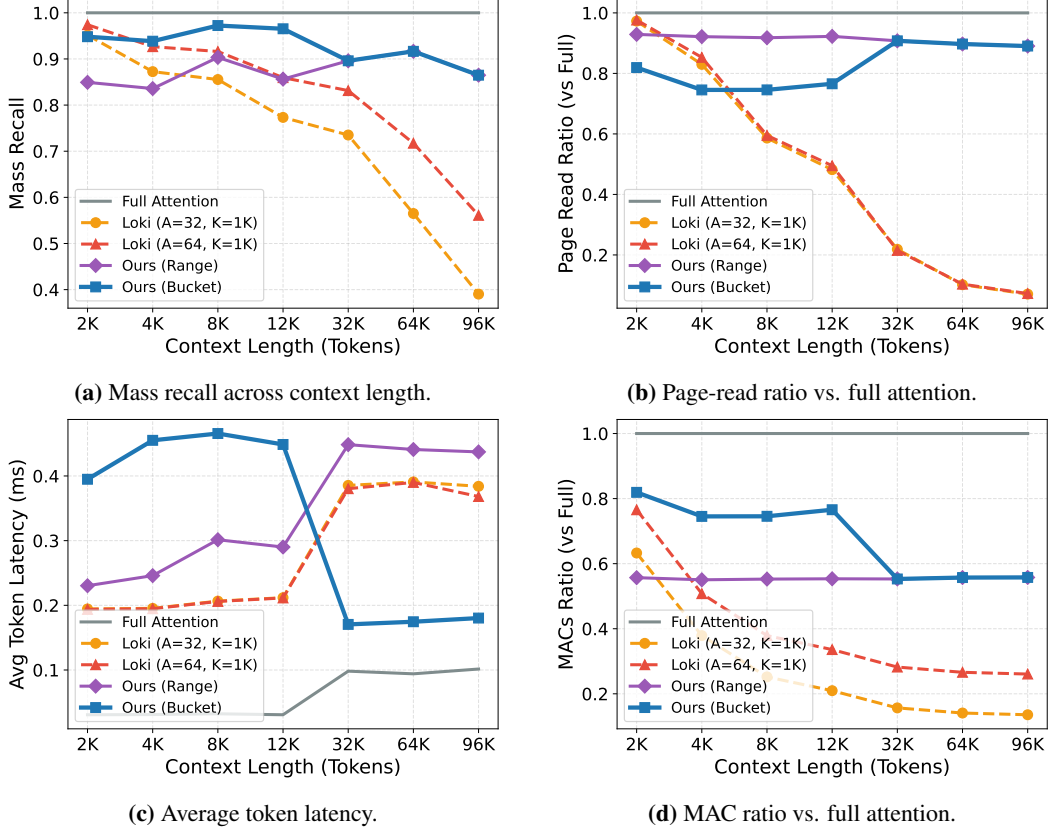
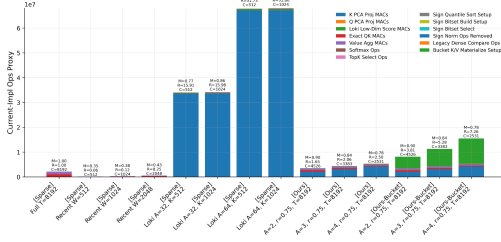


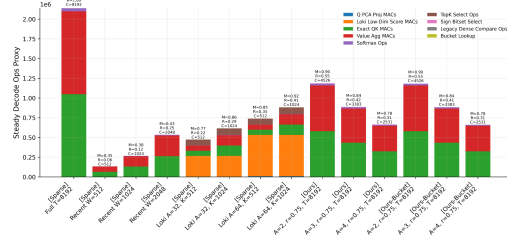
Figure 2: Scaling results from 2K to 96K. PQ-SIFT-Bucket provides a favorable quality-latency operating point among the evaluated methods by combining high mass recall with block-friendly page access. The key trend is that reducing page reads alone is not sufficient: the useful regime is where page reduction is achieved without collapsing mass recall.

The perplexity check is consistent with this interpretation. Full attention obtains PPL 10.38 on the sampled WikiText-2 test evaluation. PQ-SIFT sign/range variants with $r = 0.75$ stay close to this reference, with PPL values in the low range of approximately 10.80–11.39 for $A = 2$ to $A = 5$. Loki with $K = 1024$ is also usable but slightly worse in this check, around 11.99–12.12 for $A = 32$ and $A = 64$. In contrast, aggressive recent-window baselines degrade sharply, e.g., PPL 153.18 for $W = 2048$ and much larger values for smaller windows. This sanity check supports the central claim: PQ-SIFT is not merely pruning for speed; with a moderate r , it preserves enough attention signal to remain close to full-attention language-model behavior.

Figure 3 explains why the scaling result cannot be understood from final attention FLOPs alone. The stacked bars decompose the total work into projection/setup, low-dimensional scoring, candidate selection, exact QK, value aggregation, softmax, and materialization-related terms. In Loki, the visually dominant blue and orange portions indicate that the candidate-generation path still resembles a dense scan over the context: keys are projected/scored and then sorted or selected before exact attention can begin. In PQ-SIFT and PQ-SIFT-Bucket, the work shifts toward cheaper metadata tests and block materialization. This is the main systems insight of the experiment. A sparse attention method should not only reduce the final red/green exact-attention portions; it should also make the pre-attention selection path cheap enough that the saved arithmetic is not lost before the actual attention kernel runs.

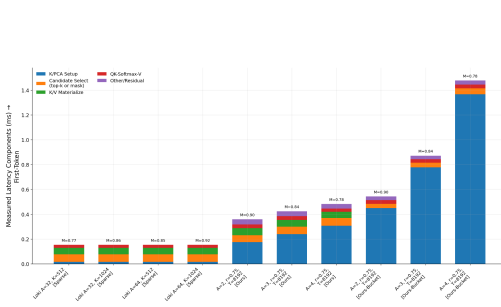


(a) Current implementation components at 8K.

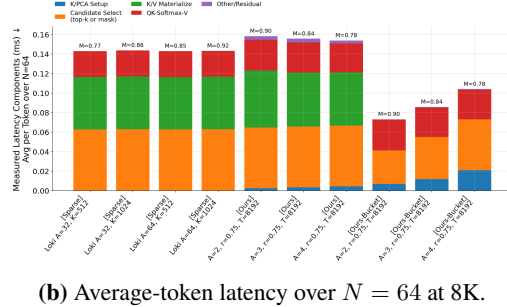


(b) Steady decode components at 8K.

Figure 3: Compute component analysis. The candidate-generation path is a first-class cost. The main visual pattern is that Loki bars are dominated by projection/scoring style work, while PQ-SIFT-Bucket trades dense ranking for metadata-based block admission and page-aligned fetch.



(a) First-token latency components at 8K.



(b) Average-token latency over $N = 64$ at 8K.

Figure 4: Latency component breakdown. Setup cost matters for the first token, whereas the average-token view highlights amortized metadata usage and block-friendly candidate fetch. The relative bar height and composition show that shorter bucketed bars are associated with lower irregular materialization overhead from page-aligned candidate blocks.

5 Analysis

5.1 Latency amortization

Figure 4 shows the difference between first-token cost and amortized decode cost. The first-token path can include K/PCA setup and metadata preparation, so it is a stress test for one-time overhead. The $N = 64$ average is closer to a realistic generation workload, where metadata and layout decisions are reused across many produced tokens. The key comparison is the relative height and composition of the bars. Non-bucketed variants can spend noticeable time on K/PCA setup and K/V materialization because selected tokens are more scattered. Bucketed retrieval reduces this overhead by fetching groups of tokens that share the same block decision. Therefore, even if a bucketed method admits slightly more candidates than an ideal token-level selector, it can still be faster because it converts irregular gathers into simpler block reads.

5.2 Quality-efficiency control by r and A

Figure 5 visualizes the central control knob. Increasing r admits more opposite-sign or range-expanded candidates, so mass recall rises but the number of final full-attention candidates also grows. This gives PQ-SIFT a useful deployment knob: a latency-critical setting can choose a smaller r , while a quality-critical setting can move right on the curve and approach full attention. The number of PCA axes A changes the curve shape. More axes can improve selectivity because the metadata test has more geometric information, but too many axes can also make the filter brittle if r is not increased. Thus, A and r should be tuned together rather than interpreted independently.

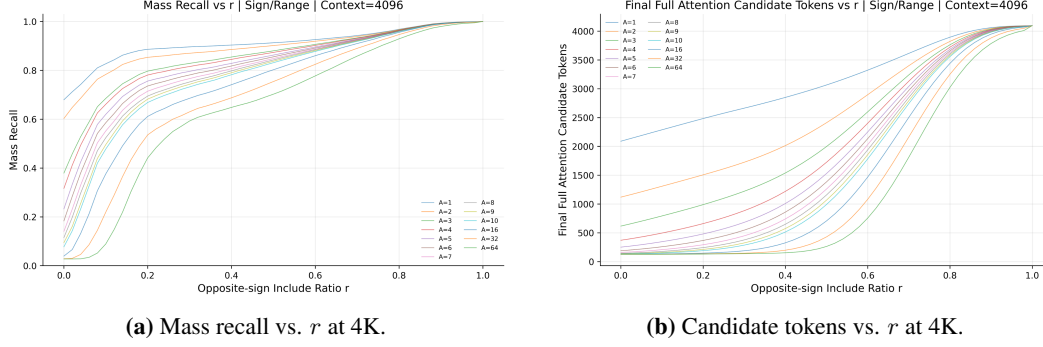


Figure 5: Quality-latency control. The inclusion ratio r directly trades attention mass recall against candidate-set size. Increasing r relaxes the sign/range filter, recovering more exact-attention mass while admitting more final-attention candidates. The useful operating region is the portion of the curve where mass recall rises sharply before the candidate count approaches full attention, making r a practical deployment knob for balancing quality and latency.

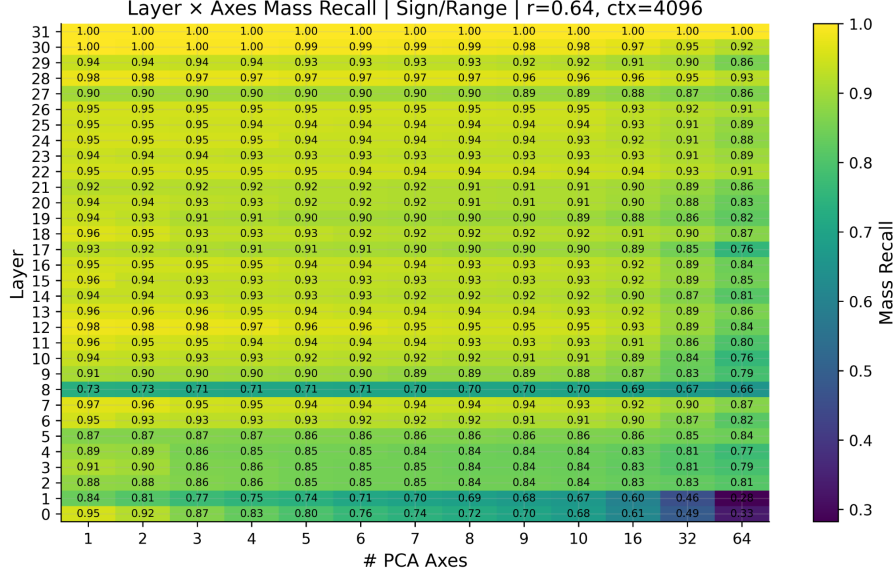


Figure 6: Layer $\times$ PCA-axis sensitivity. At $r = 0.64$ and 4K context, early layers and layer 8 are more sensitive to aggressive projected-sign filtering than most later layers.

5.3 Layer-wise sensitivity

The layer-by-axis heatmap in Figure 6 shows that a single global sparse rule is not uniformly safe. Most middle and late layers preserve high mass recall under sign/range filtering, but early layers and layer 8 are more sensitive. This is important because an average mass-recall number can hide layer-local failure modes. A practical deployment would therefore be layer adaptive: sensitive layers can use full attention or conservative thresholds, while robust layers can use more aggressive block filtering. The heatmap also explains why PQ-SIFT is better treated as a system policy than as a fixed mathematical approximation; the best policy can vary by layer, head, context length, and desired quality target.

5.4 Limitations

The current evidence is strongest at the attention and systems-measurement level rather than at full downstream task evaluation. We therefore avoid claiming universal long-context reasoning improvement. The method can also be less attractive at short context or first-token decoding, where

metadata setup is not amortized and full attention is already affordable. Another limitation is that block-level admission is intentionally coarse. It improves memory behavior, but it may read some tokens that a perfect token-level selector would skip. Finally, the current prototype is not yet a fully fused kernel; production deployment should fuse metadata lookup, candidate compaction, block fetch, and exact attention. These limitations do not undermine the main result, but they clarify the intended scope: PQ-SIFT-Bucket is most compelling for repeated long-context decode where memory movement and candidate-generation overhead dominate.

6 Conclusion

PQ-SIFT reframes long-context sparse attention as a joint retrieval, memory, and exact-attention problem. Instead of performing dense low-rank scoring followed by Top- K selection, it uses projected-query sign/range metadata to admit or reject candidate blocks. The bucketed version improves the match between sparse retrieval and GPU memory behavior by fetching 64-token chunks. Across the final twelve visual panels, PQ-SIFT-Bucket shows a practical high-fidelity operating point for long-context decode: it preserves more attention mass than the comparable Loki setting at long context, avoids the worst failure mode of recent-window sparsity, and exposes a tunable r - A trade-off for different latency targets. The broader lesson is that sparse attention should be evaluated as an end-to-end decode pipeline. A candidate selector that is accurate but expensive, or cheap but quality-destroying, is not enough. Future work should evaluate the method on downstream long-context tasks and implement a fused GPU kernel so that metadata filtering, block fetch, and exact attention can be optimized together.

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. URL: <https://arxiv.org/abs/1706.03762>.
- [2] Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020. URL: <https://openreview.net/forum?id=rkgNKkHtvB>.
- [3] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020. URL: <https://arxiv.org/abs/2004.05150>.
- [4] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020. URL: <https://arxiv.org/abs/2007.14062>.
- [5] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with performers. In *International Conference on Learning Representations*, 2021. URL: <https://arxiv.org/abs/2009.14794>.
- [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, 2022. URL: <https://arxiv.org/abs/2205.14135>.
- [7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. URL: <https://arxiv.org/abs/2309.06180>.
- [8] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. URL: <https://arxiv.org/abs/2306.14048>.

- [9] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, 2024. URL: <https://arxiv.org/abs/2309.17453>.
- [10] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. URL: <https://arxiv.org/abs/2404.14469>.
- [11] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. *arXiv preprint arXiv:2410.10819*, 2024. URL: <https://arxiv.org/abs/2410.10819>.
- [12] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference. In *International Conference on Machine Learning*, 2024. URL: <https://arxiv.org/abs/2406.10774>.
- [13] Luka Ribar, Ivan Chelombiev, Luke Hudlass-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. Sparq attention: Bandwidth-efficient llm inference. In *International Conference on Learning Representations*, 2024. URL: <https://arxiv.org/abs/2312.04985>.
- [14] Prajwal Singhania, Siddharth Singh, Shwai He, Soheil Feizi, and Abhinav Bhatele. Loki: Low-rank keys for efficient sparse attention. In *Advances in Neural Information Processing Systems*, 2024. URL: <https://arxiv.org/abs/2406.02542>.
- [15] Abhimanyu Dubey et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL: <https://arxiv.org/abs/2407.21783>.
- [16] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016. URL: <https://arxiv.org/abs/1609.07843>.

A Appendix: Team contributions

Choi Minseok. Led the problem formulation, PQ-SIFT design, baseline review, long-context sparse attention experiments.

Geonhee Jang. Contributed to project discussion, experiment organization, figure generation, systems analysis, and final report writing.